Software Engineer – Data & Analytics [Take-Home Test]

Cross-Portfolio Revenue Attribution & Forecasting Pipeline

Forethought

We believe the true essence of this role extends far beyond writing ETL scripts and SQL queries. In crafting this exercise, we've designed it to reflect the authentic challenge you'd face here: messy sources, ambiguous schemas, and stakeholders who need answers yesterday — and who don't read repos.

As practicing engineers ourselves, we understand that no one is an expert in every tool or cloud platform. The reality of our profession is that we constantly navigate unfamiliar data sources, learn new tools, and solve complex problems through research, experimentation, and pragmatic decision-making. This exercise honors that reality.

We embrace the modern landscape where AI-powered tools like GitHub Copilot, ChatGPT, Claude, and Cursor have become genuine allies. We strongly encourage you to use them, just as you would in your daily work. In fact, we assume you will — which changes what we're actually testing. Generating a repo that hits every requirement is cheap now. So we're not grading whether the pipeline exists.

We're grading whether the numbers are true, whether your judgment on the messy parts is defensible, and whether you can turn the result into something a business leader can act on. Those are the parts an agent can't do for you.

When evaluating your submission, we'll focus on your engineering *judgment* — how you structure your data, how you handle the joins that go wrong quietly, the reasoning behind your attribution choices, and how clearly you can explain what the data says to someone who isn't technical. We care about your authentic problem-solving process, not a perfect implementation. Take this as a chance to show us how you think, learn, and create. Document your journey, explain your choices, and make pragmatic decisions about where to focus. We're not looking for a perfect solution — we're looking for a trustworthy one you can defend.

Overview

Build an end-to-end analytics pipeline that ingests content performance data, portfolio company revenue data, and user journey/attribution data, then produces a revenue attribution model linking content channels to portfolio revenue. Include a basic forecasting layer using SQL or Python. Finally, turn your findings into a one-page brief a non-technical executive could act on.

Business Context

Alpha Tango's media arm (5M+ social followers, 100M+ monthly views) is the acquisition engine for its portfolio companies. Content drives traffic, traffic drives signups, signups drive revenue across portfolio businesses. But nobody can answer the question: **"Which content is actually driving revenue, and how much?"**
Marketing says social is the growth lever. The portfolio companies say organic and referral. Leadership wants data, not opinions. You're building the pipeline that connects the dots from content → traffic → signup → revenue — and then you're telling leadership what to do about it.

Starter Data

You'll receive a starter repository with pre-generated sample data files across 4 sources:

Source	Format	Records	Description
Content Performance	CSV	~1,370	Content metrics by channel — views, clicks, UTMs
User Signups	JSON	~6,170	Signup events with attribution (first/last touch, UTMs)
Portfolio Revenue	CSV	~27,070	Monthly revenue per user per company
Campaign Metadata	CSV	30	Campaign details — channel, budget, dates, target company

Channels

YouTube, Twitter/X, Instagram, Newsletter, Podcast, Blog (+ organic & referral in signups)

Portfolio Companies

ID	Name	Industry	Avg Revenue/User
CTC-001	GreenLeaf Landscaping	Home Services	~$120
CTC-002	BrightPath Education	Education	~$85
CTC-003	QuickFix Auto Repair	Automotive	~$210
CTC-004	Summit Dental Group	Healthcare	~$165

Known Data Quality Characteristics

- ~8% of content records have missing utm_source
- ~12% of users have missing UTM source
- ~15% of users have missing campaign UTMs
- Some users have conflicting first_touch_channel and last_touch_channel
- Some users have a referral_source that doesn't match their UTM data
- Revenue spans multiple months per user (recurring vs one-time)

Data is deterministic (seeded random). You can regenerate anytime with python generate_data.py.

A word to the wise: the dirty ~12–15% of records is not noise to be discarded — it's the exercise. Your two attribution models will agree on the clean users and disagree on the messy ones. How you resolve those users is most of what we're grading. And the revenue table has many rows per user; naive joins will fan out or double-count and hand you a number that's wrong but plausible. Reconcile your totals.

Functional Requirements

MUST — Data Ingestion & Loading

- Python scripts to ingest all sources into a raw/staging layer
- Data validation: UTM consistency checks, referential integrity (user_ids exist across tables), date-range validation
- Handle messy attribution data — some users have no UTM params, some have conflicting first/last touch. Don't silently drop them; make them a visible, countable category.
- Load into BigQuery (or DuckDB/PostgreSQL for local dev)

MUST — Transformation & Modeling (dbt)

- **Staging:** Clean models per source
- **Intermediate:** User-journey stitching — join signup attribution to revenue records. Resolve conflicting attribution and **document your logic** for handling missing/conflicting UTMs.
- **Mart models:**
  - fct_attributed_revenue — revenue records enriched with attribution channel, campaign, content_id
  - fct_content_performance — content metrics aggregated by channel and time period
  - fct_campaign_roi — campaign spend vs attributed revenue
  - dim_channel, dim_company, dim_user_cohort (signup-month cohort), dim_time
- **Two attribution models** (document the trade-offs):
  - Last-touch attribution
  - One other of your choice (first-touch, linear, time-decay, or position-based)
- dbt tests on attribution logic, referential integrity, and metric consistency — **including at least one test that fails if attributed revenue exceeds total revenue.**

MUST — Analytics

SQL queries answering:

1. **Channel revenue ranking:** Which content channels drive the most attributed revenue? *Under both attribution models — show how the answer changes.*
2. **Campaign ROI:** For each campaign, spend vs attributed revenue. Rank by efficiency. *(Watch the double-count: users touch multiple campaigns, revenue recurs.)*
3. **Customer Acquisition Cost (CAC):** CAC by channel per company.
4. **Cohort retention:** How does revenue retention look for users acquired via different channels?
5. **Revenue trend:** Month-over-month growth by channel, with a simple trailing-average forecast for the next 3 months.

MUST — Forecasting

- A simple revenue forecast for the next 3 months per company — SQL-based (moving averages, linear-regression approximation) or Python-based (statsmodels, simple

exponential smoothing).
- Document your approach, its limitations, and when it would break down. We expect this to be simple; the differentiation is not here, so don't over-invest.

MUST — The One-Pager *(this is the part most people skip, and the part we weight most heavily for this role)*

Leadership doesn't read repos. Alongside your code, produce a **single artifact** — a one-page memo **and** a 3-minute Loom — aimed at a non-technical executive (think a portfolio-company CEO, not an engineer). It must answer three things and nothing else:

1. **Your top 2 findings.** Which channels actually drive attributed revenue, and how confident are you? Lead with the answer, not the methodology.
2. **The decision the data implies.** Where should the next marketing dollar go — and what should we *stop* doing?
3. **What you don't trust.** Where is the data thin or ambiguous, and where did your two attribution models disagree enough that the answer depends on which one you believe?

Constraints:

- One page max, & 3 minutes max. Over length is a fail, not a bonus.
- No jargon a CEO wouldn't use. Define "last-touch" in a sentence if you use it; never mention a table name.
- **Put a number on it.** "Social drives the most revenue" is an opinion. "YouTube drives ~$X of attributed revenue, roughly N× the next channel under last-touch" is a finding.
- It's good, not bad, to say "the data can't answer this yet — here's what I'd need."

OPTIONAL — Stretch Goals

- A deployed dashboard (could be built or any dashboarding solutions) showing attribution comparison (last-touch vs your chosen model)
- Anomaly detection: flag months where a channel deviates significantly from trend
- Data-lineage documentation end-to-end
- Pipeline orchestration with an Airflow/Prefect DAG

Technical Requirements

- **Pipeline:** Python 3.10+
- **Transformations:** dbt
- **Warehouse:** BigQuery preferred. DuckDB or PostgreSQL acceptable.
- **Forecasting:** SQL window functions or Python (pandas/statsmodels/scipy)
- **SQL:** Analytical queries in standard SQL

Expected Deliverables

1. Git repository with all code
2. Sample data files (included in starter repo)
3. Python ingestion scripts
4. dbt project with full model layering and tests
5. Attribution model implementation with comparison output
6. Forecasting implementation with documented methodology
7. SQL analytics queries with sample output
8. **The One-Pager** (memo or Loom link) — see the MUST above
9. README.md with: setup instructions, architecture diagram, data model documentation, attribution model trade-off analysis, forecasting methodology & limitations, assumptions & trade-offs, future improvements

Evaluation Criteria

We've weighted this toward the parts AI can't do for you. Completing the pipeline is expected; explaining and defending it is the test.

Area	Weight	What We're Looking At
Attribution Modeling & Judgment	25%	Do your models work? Can you explain the differences in output, where they diverge, and when to use each? How did you resolve the conflicting/missing-UTM users?
Analytical Depth & Storytelling	20%	Do the queries tell a story? Is the forecast reasonable? Can you explain what the data actually says — and are your numbers *correct* (reconciled, no double-count)?
Executive Communication (the One-Pager)	15%	Does it lead with the answer, quantify it, recommend a decision, and name what you don't trust — all in language a CEO can act on?
Data Modeling & dbt	20%	Model layering, naming, tests, handling of messy joins across sources.
Pipeline Quality	15%	Ingestion, validation, handling of dirty/missing attribution data.
Documentation & Reasoning	5%	Trade-off analysis, methodology, clear setup — the *why*, not just the *what*.

Submission

- Share a GitHub/GitLab repository link to **[asif@bizscout.com]**
- Ensure the repo is public, or grant access to the above email if private
- Include a working local setup — we will run your code
- Include your one-pager & the Loom link in the return email

Notes

- Focus on a working end-to-end pipeline whose numbers you can *trust*, over a perfect architecture.
- Document any shortcuts taken due to time constraints.
- Use any libraries, AI tools, agents, or anything on the internet that speeds you up.
- Prioritize functionality, testability, and readability over perfect code.
- If your two attribution models produce nearly identical rankings, that's usually a sign the messy users got dropped — go find out why.
- If you have any questions, feel free to reach out.

What We're Looking For

- With the internet at your fingertips, how fast can you think → learn → execute & deliver, and repeat the cycle.
- Can you take messy, real-world data and turn it into something a business leader can *use*?
- Do you understand the difference between "data loaded" and "data modeled" — and between "the query runs" and "the number is right"?
- Can you model attribution in a way that's useful, not just technically correct?
- Can you write SQL that tells a story, not just returns rows?
- Can you stand in front of a CEO and say what the data means in sixty seconds?
- Do you document your thinking — the *why*, not just the *what*?
- Practical problem-solving within time constraints.

A Note on AI Tools

We genuinely mean it — use Copilot, ChatGPT, Claude, Cursor, or whatever makes you productive. We care about what you deliver and how you think, not whether you typed every character. If an AI tool helps you move faster, that's a signal you know how to leverage modern tooling, which is exactly what we want.
Just know the flip side: because we assume you're using these tools, we won't be impressed that the repo exists. We'll be impressed that your attributed revenue reconciles to the penny, that you can explain why you resolved a first-touch/last-touch conflict the way you did, and that your one-pager would actually change a decision. Let the agent write the boilerplate; you own the judgment.

Good luck — see you at the finish line!
Engineering Team

Words of wisdom from Asif:
"Don't overthink, over-complicate, or over-engineer."
"How do you eat an elephant? Bite by Bite or Byte by Byte!?"